

MCP + LangGraph + LangChain

ForgePilot production architecture, observability, guardrails, and evals

Worked example: internal GitLab coding assistant (ForgePilot).

Light layout for office printers. Assumptions marked †.

Stack: LangGraph · LangChain · MCP · LangSmith · guardrails · golden datasets.

Contents

- | | |
|-------------------------------------|-----------------------------------|
| 1 How to use this brief | 6 Observability (LangSmith) |
| 2 Product and layers | Fig 8 Trace pipeline |
| Fig 1 System context | 7 Guardrails |
| Fig 2 Layered stack | Fig 9 Four chokepoints |
| 3 Production reference architecture | 8 Evals and golden datasets |
| Fig 3 Full reference + tooling map | Fig 10 Eval flywheel |
| 4 LangGraph and MCP internals | Fig 11 Dataset schema + scorecard |
| Fig 4 StateGraph control plane | 9 Path to production |
| Fig 5 MCP tool plane | Fig 12 CI/CD · Fig 13 Envs |
| 5 Sequence diagrams | Fig 14 Debug loop · Fig 15 Cost |
| Fig 6 Q&A | 10 Expose the agent · Fig 16 /mcp |
| Fig 7 Fix + draft MR | 11 Implementation sketch |
| Fig 7b MCP elicitation | 12 Assumptions vs sources |

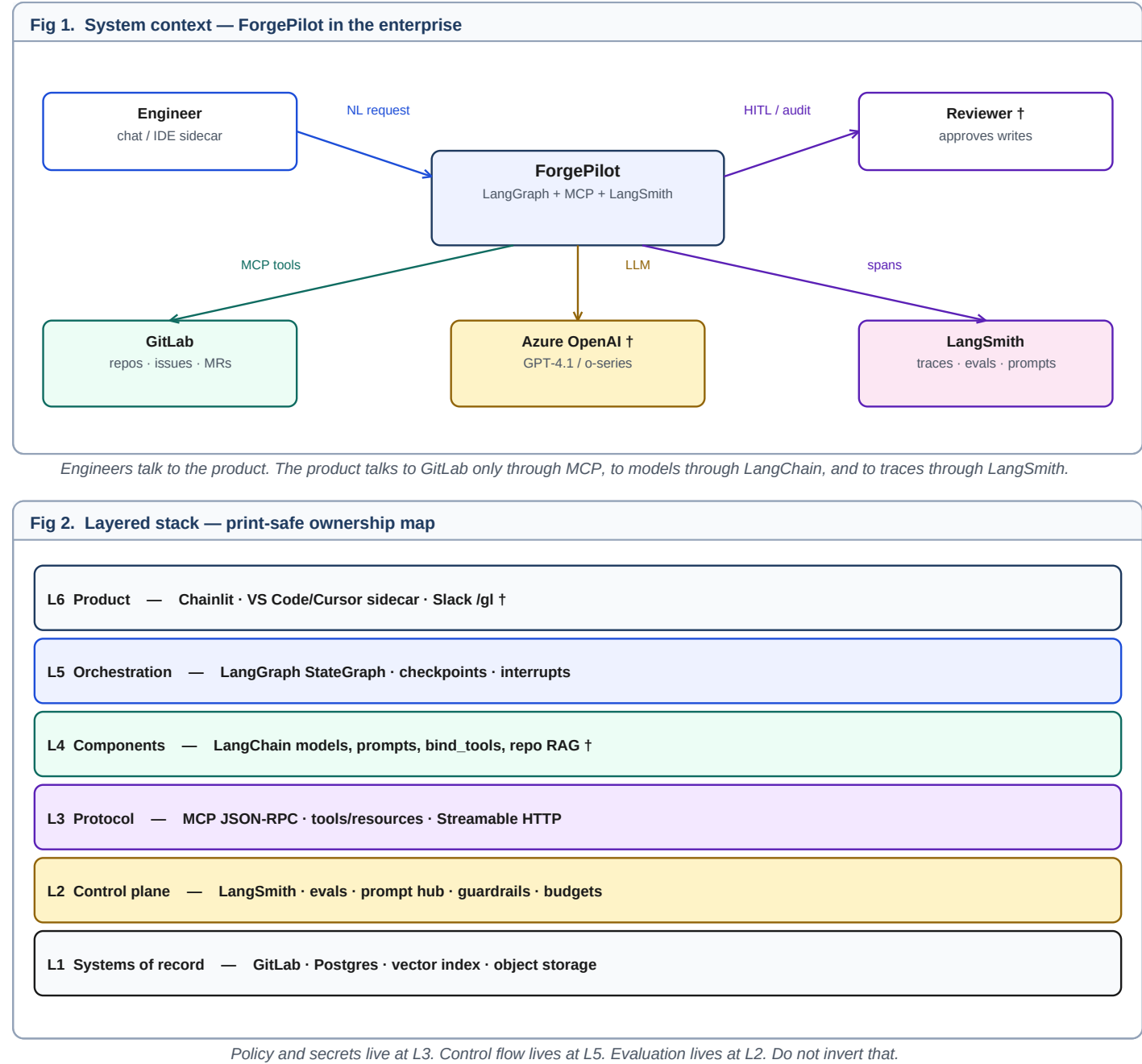
1. How to use this brief

This is a printable architecture pack for getting an MCP + LangGraph agent into production. The worked product is **ForgePilot**, an internal GitLab coding assistant: retrieve code through MCP, plan and patch in a LangGraph loop, pause for a human, then open a draft merge request. LangChain supplies model adapters and tool objects. **LangSmith** is the production control plane for traces, prompt versions, golden datasets, experiments, and online monitors. Items marked † are assumptions layered on public write-ups.

Print notes: letter, light backgrounds, dark type, hairline rules. Do not photocopy the cover as a dark plate — there isn't one. Diagrams are vector; they survive 600 dpi office printers.

2. Product and layers

ForgePilot answers code questions, writes docstrings, and opens draft MRs. It never pushes to main. MCP holds GitLab credentials and policy. LangGraph holds control flow. LangSmith holds the truth about what happened.

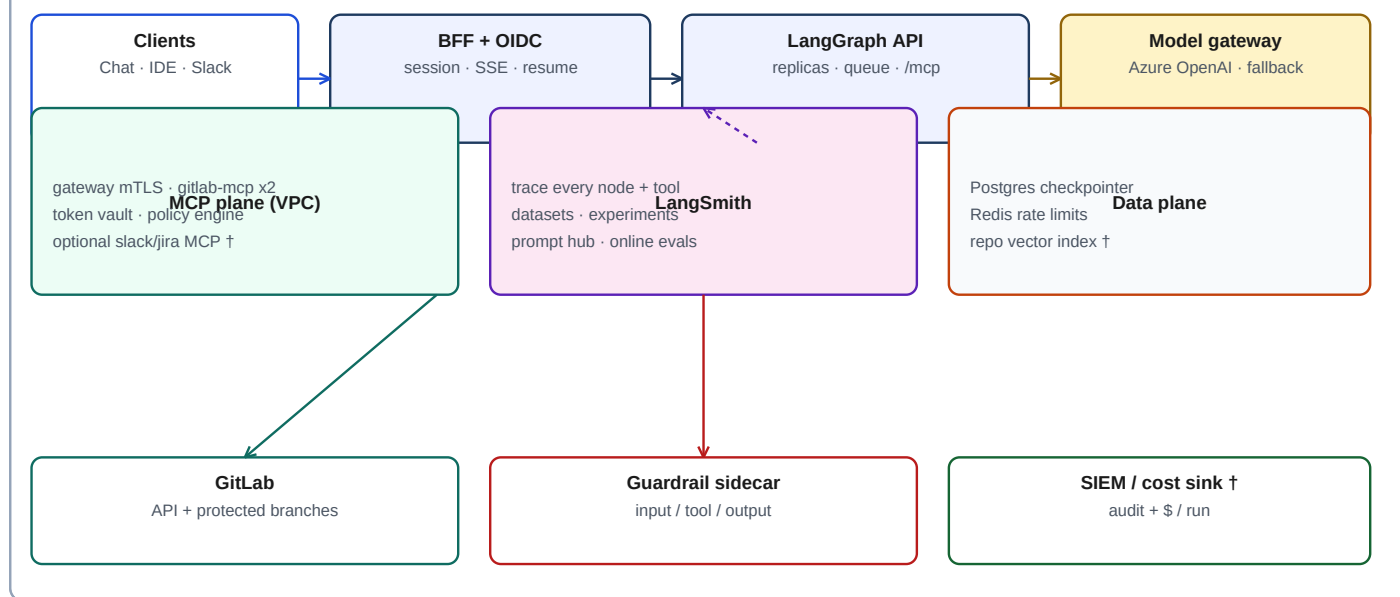


3. Production reference architecture

Ship as two planes. The brain (LangGraph API, models, LangSmith exporter) can sit in Azure. The hands (gitlab-mcp, token vault) stay on the corporate network. A BFF terminates OIDC and streams tokens. A guardrail sidecar sits on input, tool, and output paths.

Postgres checkpoints make interrupts and crashes resumable.

Fig 3. Production reference architecture †



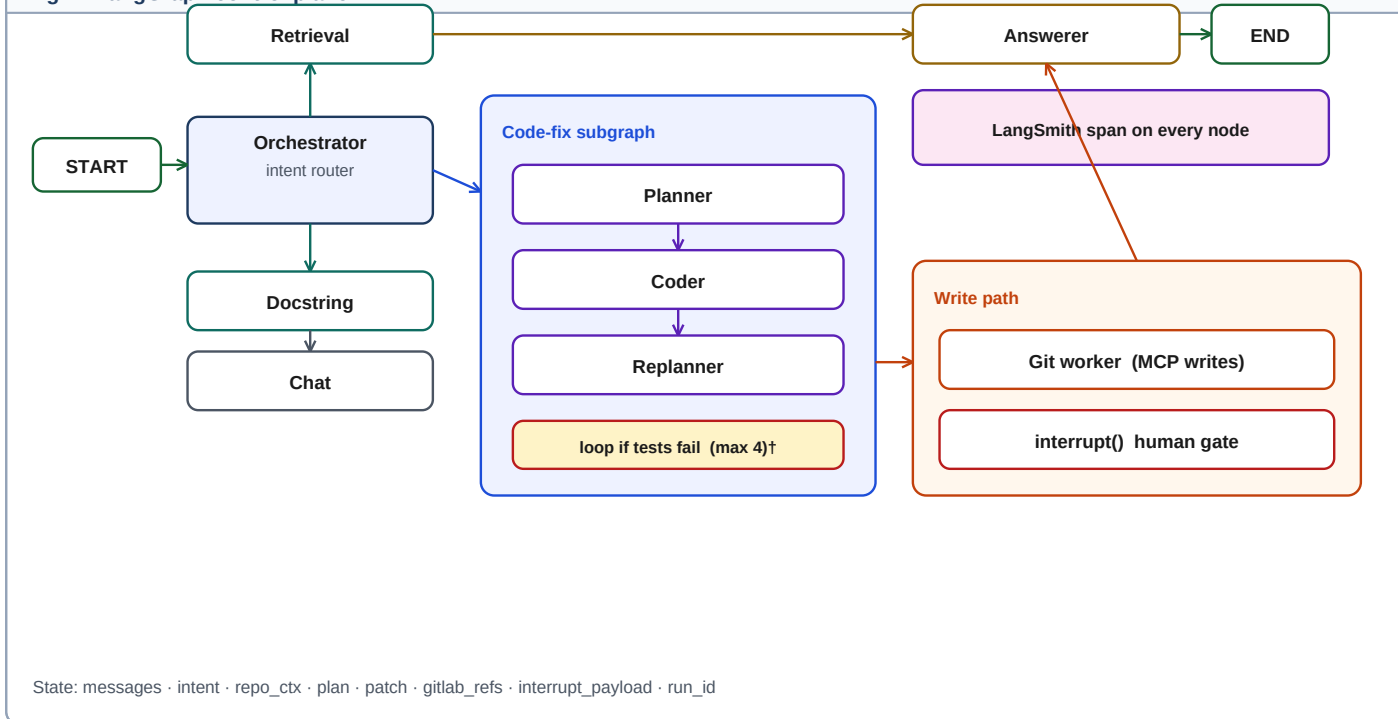
Split-plane deploy. Cloud reasoning, on-prem systems of record, shared observability.

3.1 Tooling map — what actually gets you to production

Need	Tool	Role
Orchestration	LangGraph + Agent Server / LangGraph Platform	graphs, checkpoints, /mcp expose, replicas
Components	LangChain	models, prompts, BaseTool, MCP adapter
Tools / data	MCP servers (GitLab, Slack, Jira †)	isolated I/O, schema, policy
Observability	LangSmith	traces, feedback, cost, prompt hub
Evals	LangSmith Datasets + Experiments	offline champion/challenger
Online quality	LangSmith monitors / online evals †	sample prod, alert on drift
Guardrails	sidecar + MCP policy + interrupt()	PII, inject, secret leak, write gate
Identity	OIDC + vault (KMS)	user JWT → GitLab token; graph never sees PAT
State	Postgres checkpointer	resume after crash or HITL
Limits	Redis + graph middleware	rate, \$ / run, loop cap
Metrics / pages	Grafana + SIEM †	SLO burn, audit export
Prompt mgmt	LangSmith Prompt Hub	pin hash in prod; no ad-hoc strings
Local IDE reuse	Agent Server /mcp	Cursor/Claude call ForgePilot as a tool

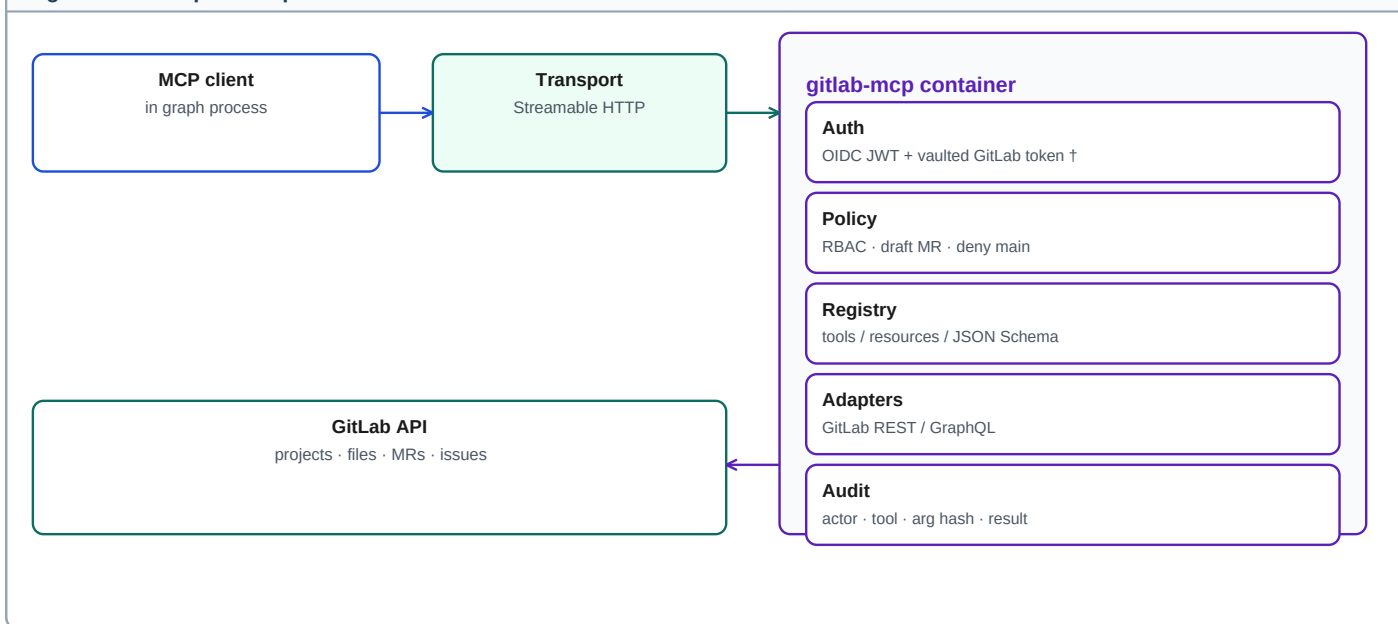
4. LangGraph and MCP internals

Fig 4. LangGraph control plane



Supervisor plus cyclic code-fix subgraph. Write path is the only place write tools are bound.

Fig 5. MCP tool plane — process isolation



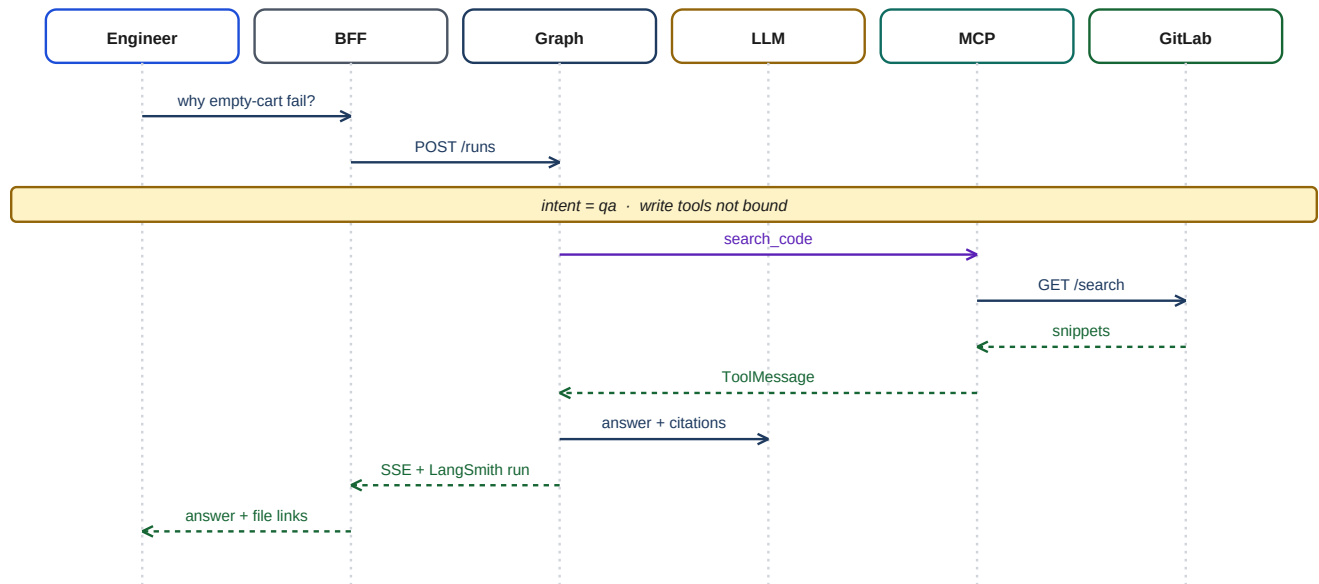
Last line of defense is the MCP server: it can rewrite `draft=true` and 403 protected branches even if the model asks otherwise.

4.1 Tool allowlist

Tool	Side effect	Gate
<code>list_projects</code> , <code>get_file</code> , <code>search_code</code> , <code>get_tree</code>	read	repo allowlist only
<code>list_issues</code> , <code>get_issue</code> , <code>list_mrs</code>	read	none beyond ACL
<code>create_issue</code>	write tracker	confirm project + title
<code>create_branch</code> , <code>update_file</code>	write git	<code>interrupt()</code> ; never default branch
<code>create_merge_request</code>	write git	<code>interrupt()</code> ; server forces draft

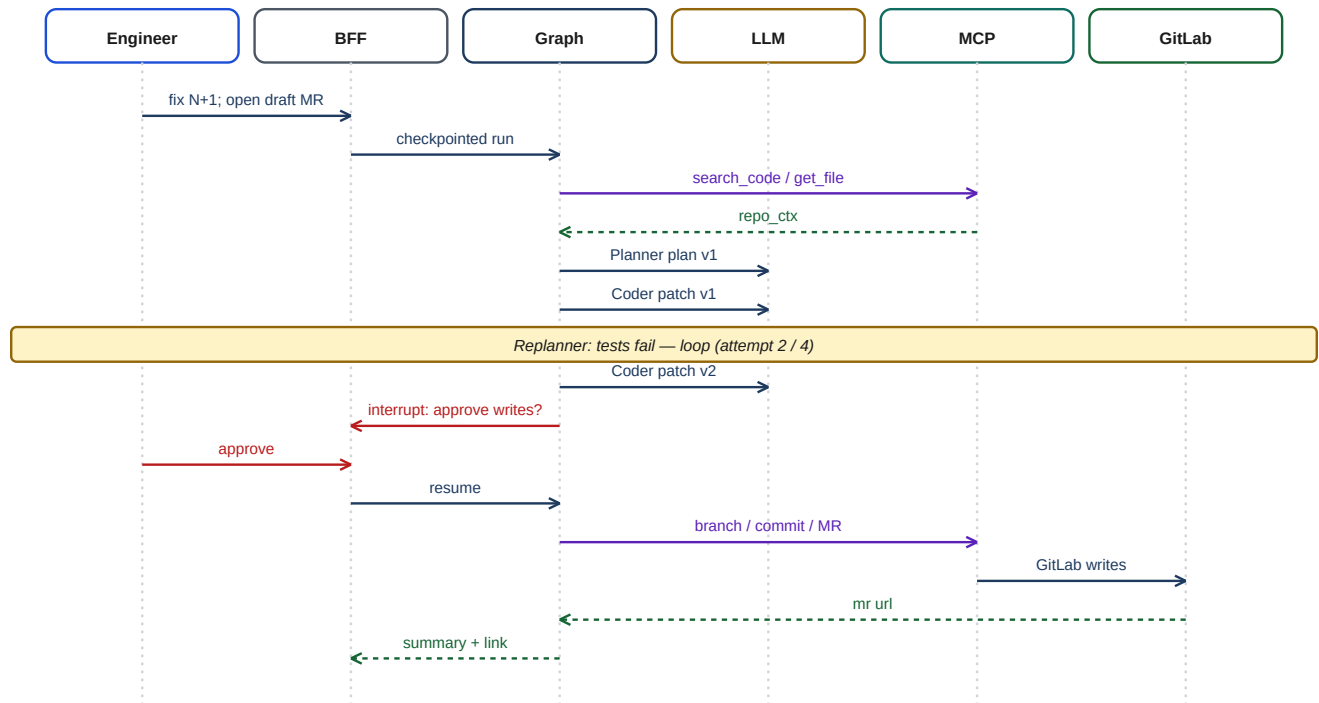
5. Sequence diagrams

Fig 6. Sequence A — read-only Q&A



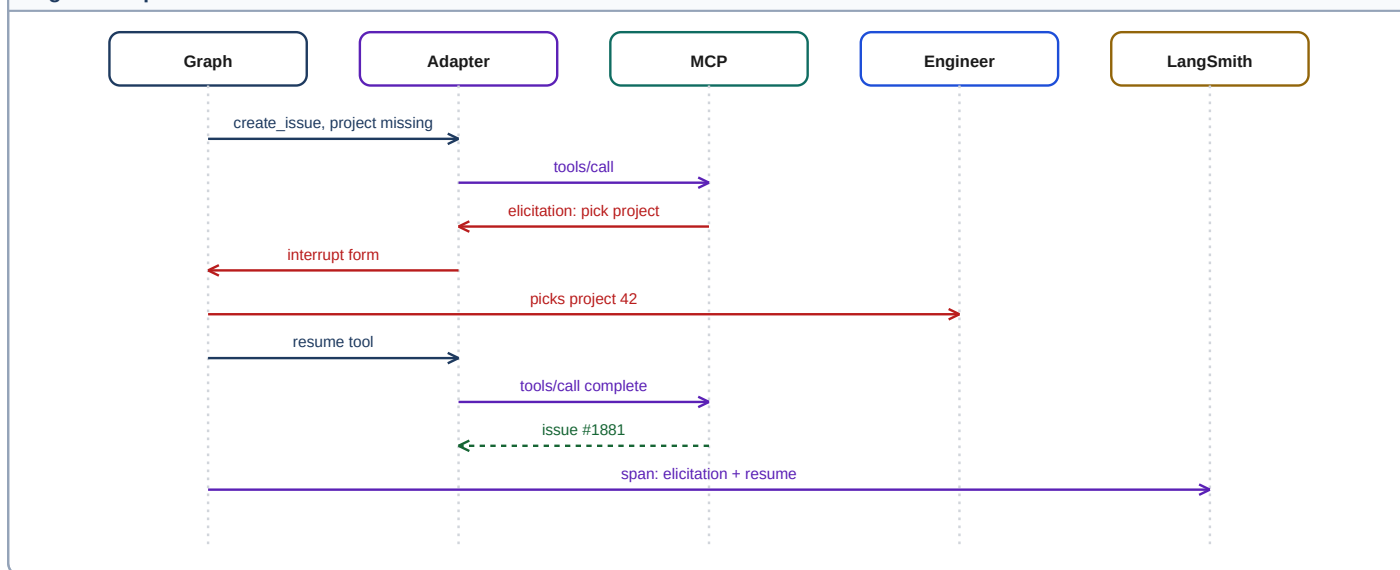
No Git worker, no interrupt. Trace still lands in LangSmith so Q&A; quality can be scored later.

Fig 7. Sequence B — fix loop, HITL, draft MR



`interrupt()` returns the HTTP run as paused. Resume is a second request. Human wait is out of the SLO.

Fig 7b. Sequence C — MCP elicitation + trace

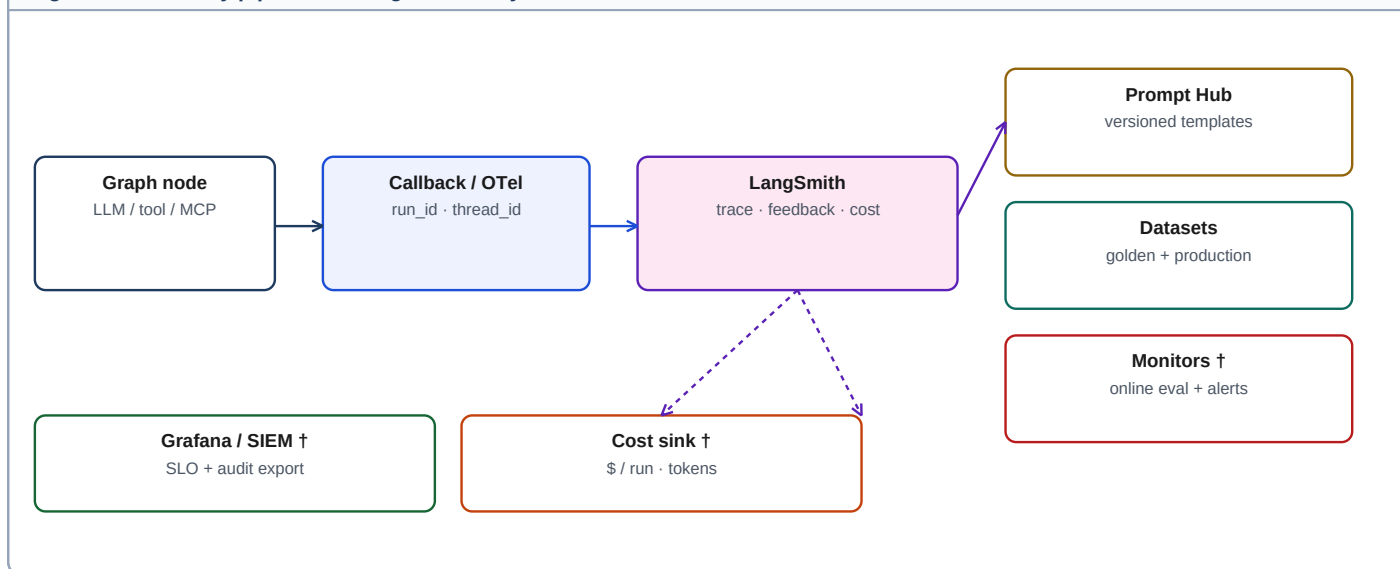


2026 MCP elicitation becomes a LangGraph interrupt. LangSmith records both halves as one run.

6. Observability with LangSmith

Treat every production run as a trace with a stable `run_id` and `thread_id`. Child spans: orchestrator, each MCP call (name, latency, error code), each LLM call (model, tokens, \$), interrupt wait, resume. Attach user feedback (thumbs, “wrong tool”, “bad MR”) as LangSmith feedback keys so online evals and golden promotion have a signal.

Fig 8. Observability pipeline — LangSmith as system of record



LangSmith is the product analytics system for the agent, not an optional debugger.

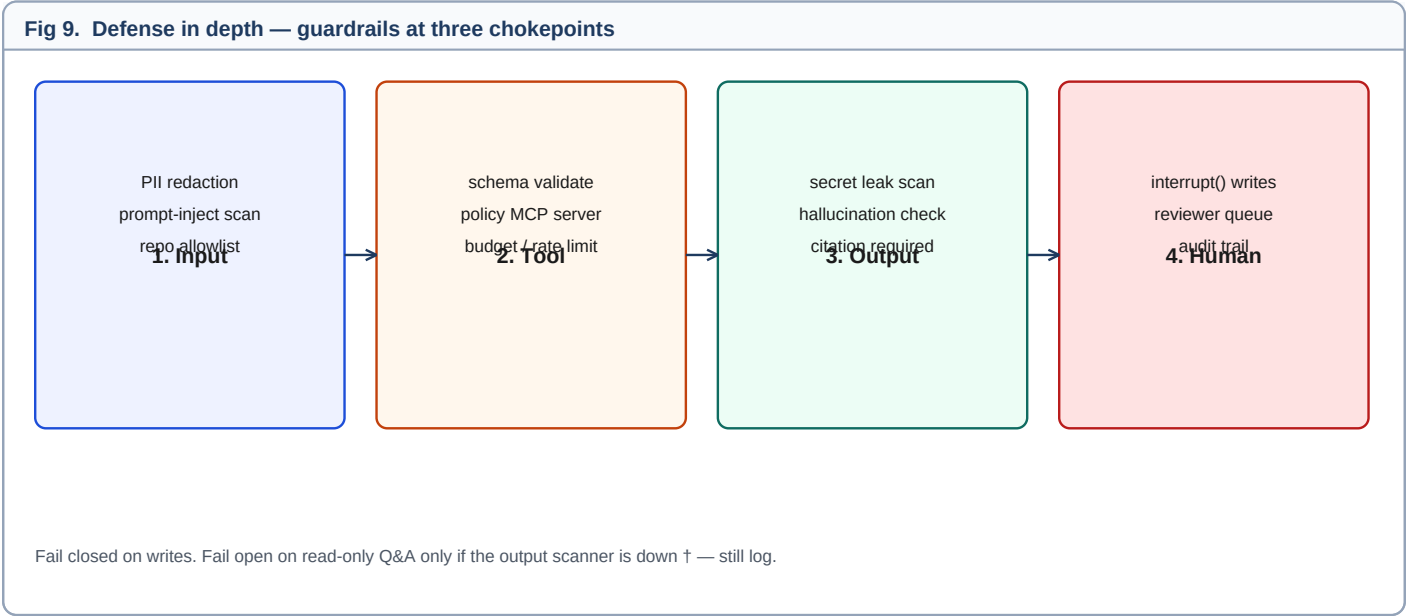
6.1 What to trace (minimum viable telemetry)

Span / field	Why
graph_node + state_delta_hash	see which node produced a bad plan
mcp.tool, mcp.error, mcp.latency_ms	separate model mistakes from GitLab/MCP outages
llm.model, prompt_hash, tokens, usd	cost and prompt regression
interrupt.reason, actor, decision	audit writes; measure HITL friction
output.citation_paths[]	grounding eval

Span / field	Why
feedback.wrong_tool / bad_mr / thumbs	promote hard cases into the golden set

7. Guardrails

Four chokepoints. Input filters keep secrets and off-repo paths out. Tool gates enforce schema and MCP policy. Output scanners block leaked tokens and uncited answers on Q&A.; Humans own every git write. Implement input/output scanners as a sidecar so they can be updated without redeploying the graph. Keep write policy on the MCP server so a compromised prompt cannot push to main.

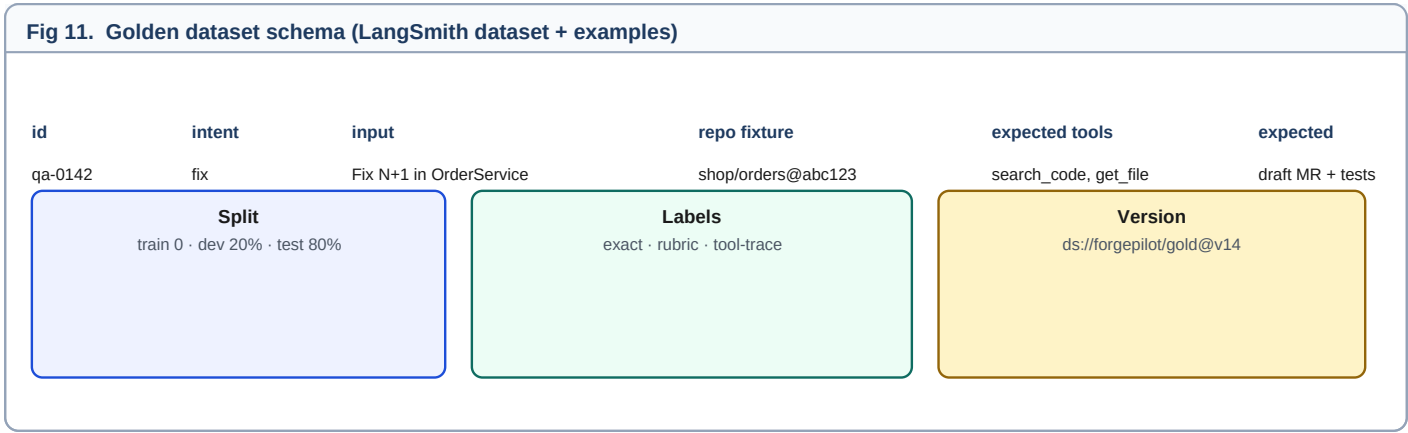
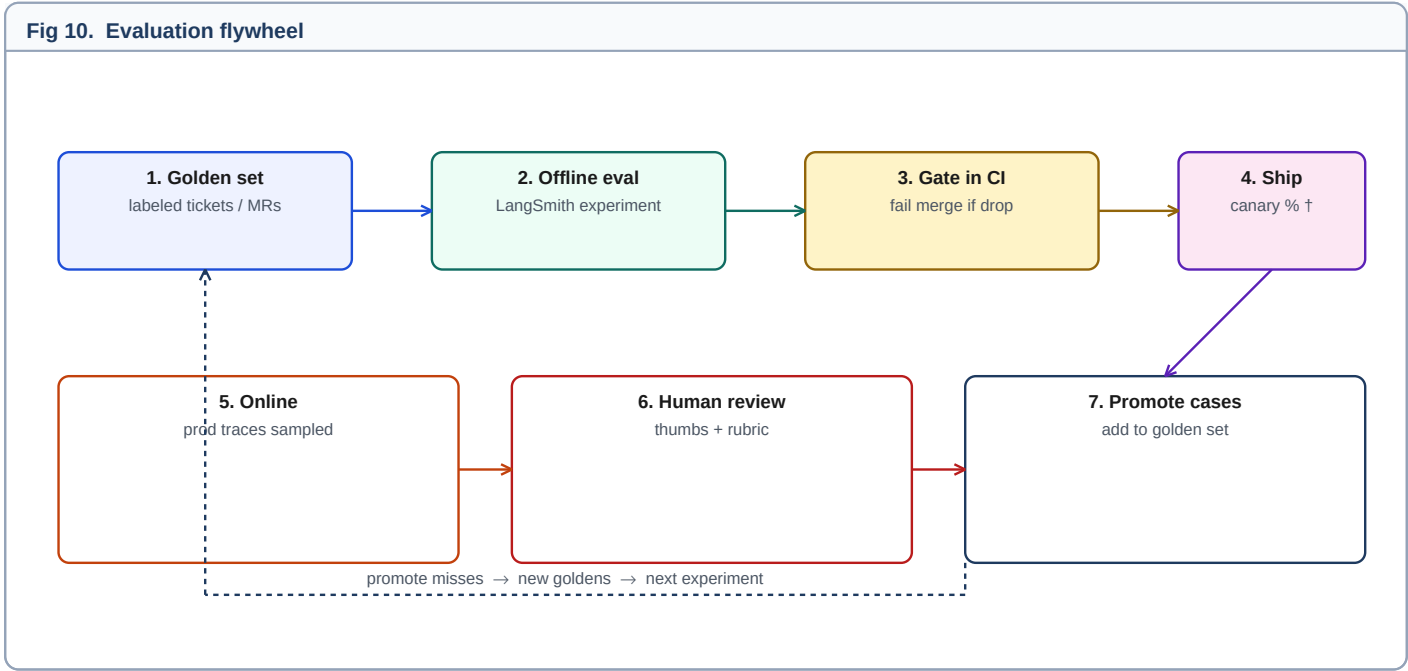


UX gates are interrupt(). Safety gates are MCP policy. Both are required.

Risk	Control	Fail mode
Prompt injection ("ignore policy, push main")	MCP deny + no write tools on qa path	closed
PII in prompt / log	redact before LLM and before LangSmith payload †	closed
Secret in generated patch	output regex + pre-commit hook on worktree	closed
Cost runaway	middleware: \$0.80 or 4 loops †	closed (ask user to narrow)
Wrong repo	allowlist from user ACL	closed
Hallucinated API	grounding eval + cite paths	open on Q&A; if cited; closed on writes

8. Evals and golden datasets

Do not ship prompt or graph changes on vibe. Build a versioned LangSmith dataset of real tickets. Each example has input, repo fixture commit, expected tool trace, and a rubric for the MR/answer. Offline experiments compare challenger vs champion. CI fails the PR if tool-call accuracy or grounded-answer rate drops more than the budget. After ship, sample production traces, have humans label misses, and promote those misses into the next dataset version. That is the flywheel.

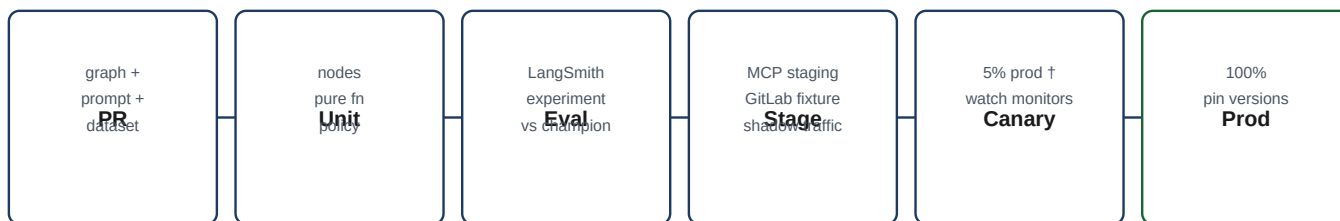


8.1 Scorecard †

Metric	How	Ship gate
Tool-call accuracy	exact match on first tool + args subset	≥ 90% on gold test
Grounded Q&A;	LLM-as-judge + citation overlap	≥ 85%
Draft-MR accept rate	human label: mergeable / needs-rework	≥ 70%
Protected-branch attempts	MCP 403 count	0 in stage/prod
\$ / successful MR	LangSmith token + tool cost	≤ \$0.50 P50
P95 Q&A; latency	trace timestamps	< 8 s
HITL time	interrupt span	measure; not a gate

9. Path to production

Fig 12. Path to production — CI/CD for agents

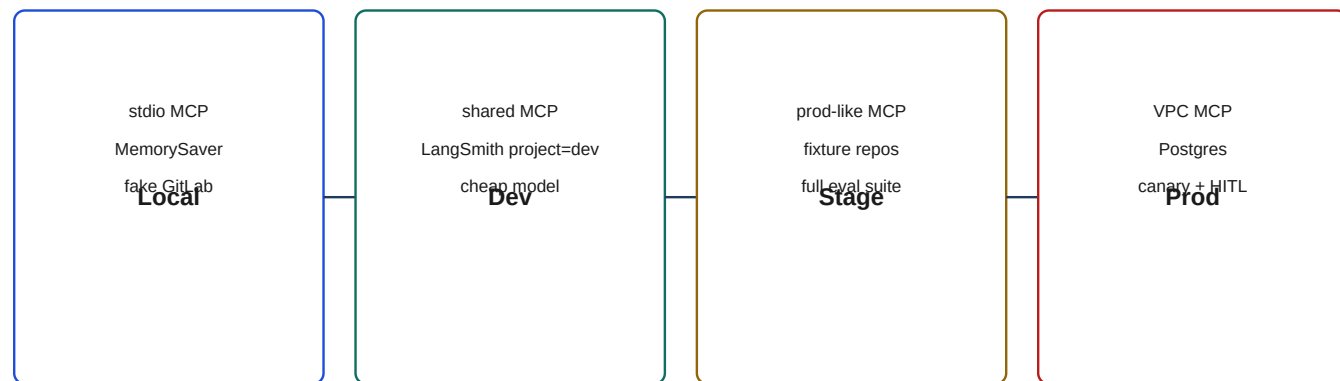


Hard gate: tool-call accuracy, grounded-answer rate, and \$ / successful MR must not regress beyond budget †.

Pin langchain, langgraph, mcp adapter, prompt hash, dataset version. No floating latest.

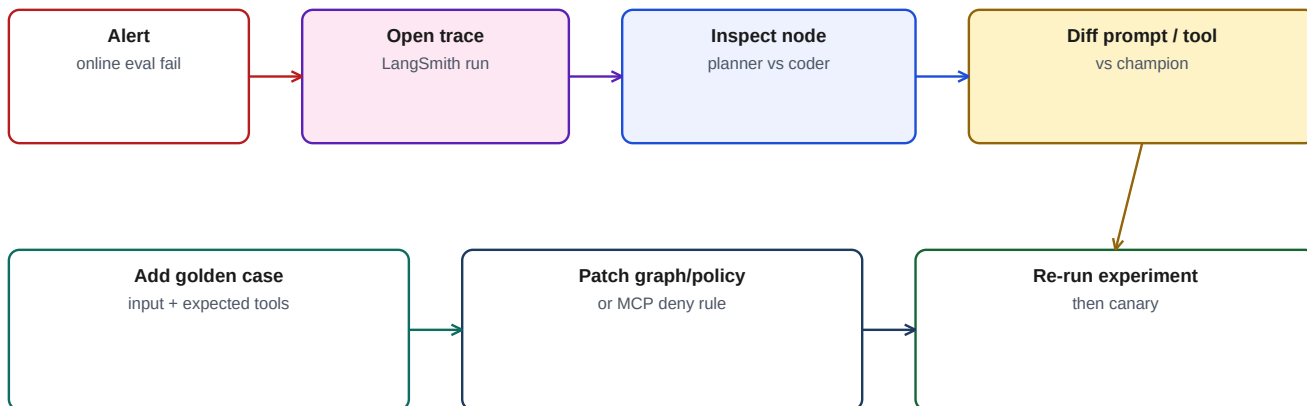
Pin library versions, prompt hash, and dataset version. Floating latest is how silent quality drops land on Friday.

Fig 13. Environments

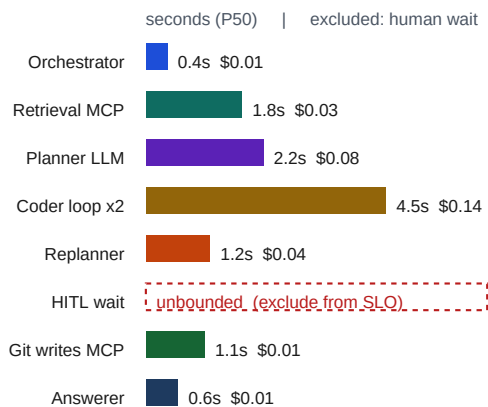


Local uses stdio MCP. Prod uses Streamable HTTP in the VPC. Never point dev graphs at prod GitLab write tools.

Fig 14. Debug loop when a bad MR is opened



A bad MR is an eval bug until proven otherwise. Add the case before patching the graph.

Fig 15. Cost and latency waterfall (typical fix+MR job) †

Budget cap †: \$0.80 / run or 4 coder loops — graph middleware aborts and asks the human to narrow scope.

Human wait excluded. Published jobs of this shape land around \$0.30–\$0.45 and 10–15 model/tool calls.

10. Expose the agent itself

Fig 16. Bidirectional MCP — consume tools and expose the agent

Same graph, two faces: a chat app and an MCP tool for IDEs.

11. Implementation sketch

```
# conceptual – auth, pooling, error mapping omitted
from langchain.mcp import MCPAdapter
from langgraph.graph import StateGraph
from langsmith import traceable

@traceable(name="git_worker")
def git_worker(state):
    decision = interrupt({"type": "approve_writes", "diff": state["patch"]})
    if decision.get("ok"):
        return call_write_tools(state)
    return {"messages": [("system", "writes cancelled")]}

async def build(checkpointer):
    async with MCPAdapter("https://mcp.gitlab.internal/mcp") as mcp:
        tools = await mcp.list_tools()
        g = StateGraph(ForgeState)
        g.add_node("orchestrator", orchestrator)
        g.add_node("retrieve", make_retrieve(tools))
        g.add_node("fix", make_fix_subgraph(tools))
        g.add_node("git_worker", git_worker)
```

```
return g.compile(checkpointer=checkpointer)
```

12. Assumptions vs sources

Grounded: LangGraph supervisor + retrieval / docstring / code-fix / git-worker; GitLab via MCP; draft MRs; Developer-role least privilege; ~10–15 calls and ~\$0.30–\$0.45 per job; MCP as team boundary; LangChain MCP adapter and elicitation-as-interrupt; Agent Server /mcp; WEBTOON WCAI on LangGraph; LangSmith as the eval/trace plane in the LangChain ecosystem.

Assumed †: name ForgePilot; AKS + VPC split; Azure OpenAI; Postgres checkpointer; OIDC vault; Slack/Jira MCP; \$0.80 run cap; numeric SLOs and scorecard thresholds; 5% canary; 5–10% prod sampling; PII redaction before LangSmith; WEBTOON shipping the MCP adapter.

- inovex — Agentic Workflows and Model Context Protocol (GitLab MCP + LangGraph).
- Towards Data Science — The Protocol That Cleaned Up Our Agent Architecture.
- LangChain docs — langchain.mcp, LangGraph Agent Server MCP endpoint, LangSmith datasets/experiments.
- LangChain customer story — WEBTOON Comprehension AI (WCAI).
- Model Context Protocol spec — tools, resources, elicitation, Streamable HTTP.

End of print edition. Replace every † before treating thresholds as policy. Diagrams are schematic, not capacity plans.